k-NN 学習方策による小型二輪ロボット E-puck の経由地点走行

— Webots シミュレーション上での AI×ロボティクス 最終課題 —

Physical AI 応用1講座 最終課題 説明資料（課題名：AI×ロボティクスによる創造的タスクの実現）

提出者: omnicampus アカウント名 YUMA888AI（ID 88585） **作成日:** 2026年10月23日 **使用ツール:** Webots R2025a／Python 3（標準ライブラリのみ）／macOS（Apple Silicon）

1. 課題の概要と目的

実機を使わず、ロボットシミュレータ **Webots** 上の小型二輪ロボット **E-puck** を Python で制御し、**教師あり学習（k近傍法, k-NN）**が各時刻の行動（前進・左旋回・右旋回）を推論することで、障害物の置かれた 2.4 m 四方のフィールド内に人が設定した **9 か所の経由地点**を順番に通過し、最終的に**緑色のゴール**へ到達する走行タスクを実現した。

本課題で確かめたかったことは次の 2 点である。

1. 合成教師データで学習した軽量な AI（k-NN）だけで、安定した「局所的な行動判断」ができるか。
2. AI が担う範囲（行動判断）と人が担う範囲（経路の設計）を明確に分けた構成で、再現性と説明可能性の高いロボット制御が組めるか。

結果: Webots 上で経由地点 9/9 を通過し、**シミュレーション時間 63.2 秒でゴール到達**（最終位置とゴール中心の距離 0.108 m、ゴール半径 0.11 m 以内）を確認した。走行の全過程をコンソールログと共に動画に収録している。

2. 提出物

提出物	内容
動画（YouTube）	タイトル `PAI最終課題_YUMA888AI`。長さ 66 秒、**等速（倍速なし）**、日本語字幕 7 本を焼き込み。Webots の 3D 画面とコンソールログを同時に収録。 URL: （YouTube アップロード後、omnicampus の指定フォームに記入）
コード一式（GitHub, public）	`https://github.com/onigiri1star/pai-final-epuck-knn-webots` ワールド・コントローラ・k-NN 実装・教師データ生成／評価スクリプト・README・字幕 SRT・本資料 PDF
本資料（PDF）	課題の概要・目的・手法・結果・工夫点・限界・考察

3. システム構成

項目	内容
シミュレータ	Webots R2025a（Nightly Build 21/10/2026、basicTimeStep 32 ms）
実行環境	macOS 27.0.1（Apple Silicon）／Python 3.14.7（Webots の Python command に絶対パスを指定）。Nightly ビルドのため、他環境で同一の走行結果になることは保証しない
ロボット	E-puck（二輪差動駆動、前方赤外線近接センサー ps0–ps7 の 8 個を使用）
フィールド	2.4 m × 2.4 m の床、四方に高さ 0.14 m の壁、障害物 2 個（赤：0.22×0.16×0.14 m を (–0.10, 0.00)、青：0.16×0.16×0.12 m を (0.34, 0.24)）、ゴールマーカー（緑の球、中心 (0.76, 0.40)、到達判定半径 0.11 m）
制御プログラム	`controllers/ai_navigation/ai_navigation.py`（Webots Supervisor コントローラ。特徴量生成 → k-NN 推論 → 左右モータ速度の決定）
AI モデル	`learned_policy.py`：純 Python の k-NN 分類器（k = 3、重み付き距離・距離逆数投票）。NumPy / scikit-learn 不使用
教師データ	`training_data.json`：6,000 例（`scripts/train_model.py` がルールベースの教師方策から乱数シード固定で合成）
評価スクリプト	`scripts/evaluate_model.py`：Webots 不要のオフライン精度検証

4. AI の設計（k-NN 学習方策）

4.1 入出力

入力特徴量（3 次元）	定義
右前方の近接度 a	ps0・ps1・ps2 の最大値から待機時のベースライン 75 を引き、325 で割って 0–1 に正規化
左前方の近接度 b	ps5・ps6・ps7 について同様
目標方位差 c	現在の経由地点への方位と機体の向きの差（$-\pi$〜π）を π で割り −1〜1 に正規化
出力（3 クラス）	**モータ指令**
forward	左右とも 4.1 rad/s（約 0.084 m/s）
turn_left / turn_right	左右の車輪を ∓2.4 / ±2.4 rad/s で逆回転（その場旋回）

4.2 教師データの合成と学習

教師方策（ルール）は「前方近接度が 0.19 以上なら障害物の少ない側へ旋回、それ以外は方位差が ±0.065 を超えたら方位差の向きへ旋回、さもなくば前進」という 3 条件で定義した。これを用いて、自由空間 42%・右前方障害物 26%・左前方障害物 26%・両側障害物 6% の比率で乱数生成した入力にラベルを付け、さらに「前進と小さな旋回の境界」を学ばせるため方位差 ±0.06 以内の自由空間サンプル 350 例を密に追加し、合計 **6,000 例**とした（`seed = 20261020` で完全に再現可能）。k-NN は学習時にサンプルを保持するだけで、推論時に距離 $d = 9(\Delta a)^2 + 9(\Delta b)^2 + 5(\Delta c)^2$ が小さい上位 3 例の 1/(0.02

+ *d*) 重み付き投票で行動を決める。障害物の項に大きな重みを与え、衝突回避が目標方位より優先されるようにした。

4.3 オフライン精度

`evaluate_model.py`（クラス均等 300 例、学習データとは別に生成）での一致率は **94.3%**（forward 100/100、turn_left 89/100、turn_right 94/100）であった。これは「教師ルールの判断を k-NN がどれだけ再現できるか」の指標であり、走行成功率そのものではない。

4.4 AI 判断の周りの安全機構

前方近接度の生値が 100 を超えた場合、AI の判断した旋回（前進判断なら障害物の少ない側への旋回）を 0.65 秒間保持する状態機械を設けた。反射的な前進／旋回の切り替え振動を防ぐための設計である（今回の走行では発動していない。§8 参照）。

5. タスク設計：経由地点を用いた往復ルート

AI が担当するのは各時刻の局所的な行動判断であり、**どこを通るか（経路）は人が経由地点として設計した**。走行距離を伸ばして AI の判断を長時間検証するため、障害物の南側を通る回廊を **2 回往復**してからゴールへ向かうルートとした（経由地点間の直線距離の合計 約 5.3 m）。



図1　フィールドと設計ルート（上から見た図、単位 m）。障害物の南側の回廊を往復し、最後に北東のゴールへ向かう。

#	座標 (x, y) [m]	到達時刻*
START	(−0.80, −0.34)	0.0 s
WP1	(−0.43, −0.43)	3.0–6.0 s
WP2	(0.12, −0.47)	9.1–12.1 s
WP3	(0.69, −0.12)	18.1–21.1 s
WP4	(0.12, −0.47)	24.1–27.1 s
WP5	(−0.43, −0.43)	30.1–33.1 s
WP6	(−0.80, −0.34)	36.1–39.1 s
WP7	(−0.43, −0.43)	39.1–42.1 s
WP8	(0.12, −0.47)	45.2–48.2 s
WP9	(0.69, −0.12)	54.2–57.2 s
GOAL	(0.76, 0.40)	**63.2 s**

表1　経由地点（到達判定半径 0.13 m）と、ログ上で「Waypoint n/9 reached」が出力された区間（*3 秒ごとのログ行の間）。

6. 実験結果（Webots 上での走行）

Webots R2025a（macOS）で等速実行し、画面を収録した。結果の要点は次のとおり。

項目	結果
経由地点の通過	**9/9**（表1の順に全て通過）
ゴール到達	**成功**：`[PAI] SUCCESS! Goal reached at t=63.2s`
最終位置／ゴールとの距離	(0.775, 0.293)／**0.108 m**（判定半径 0.11 m 以内）
走行距離・平均速度（概算）	約 5.3 m・約 0.084 m/s（経由地点間の直線距離の合計と所要時間から算出）
AI の判断（3 秒ごとのログ 22 行）	forward 21 回、turn_right 1 回（t = 39.1 s、WP6 = スタート地点で折り返す場面）。折り返し（WP3→WP4 など）の旋回はログ行の間で行われている
赤外線センサー値	全ログで 65–75（待機値付近）。障害物との接近・接触は発生していない
タイムアウト（180 s）	該当なし

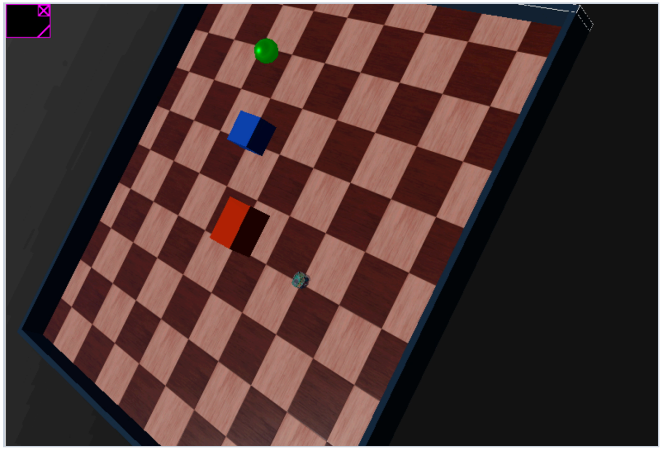


図2　走行開始直後（動画 3 秒）。左下のスタート地点から東へ進む。

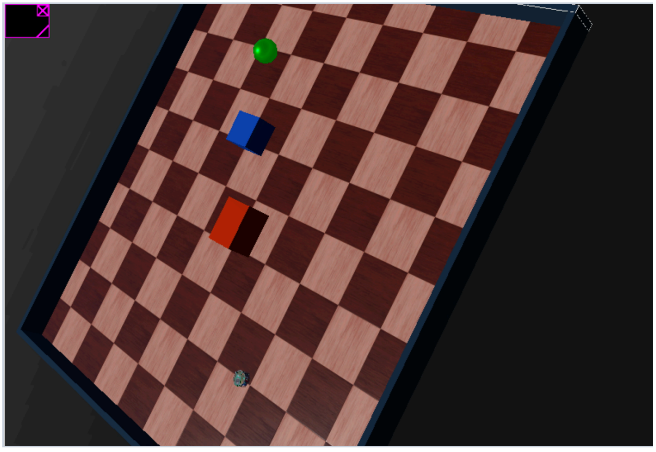


図3　スタート地点に戻って折り返す場面（動画 34 秒、ログ t = 39.1 s で AI = turn_right）。

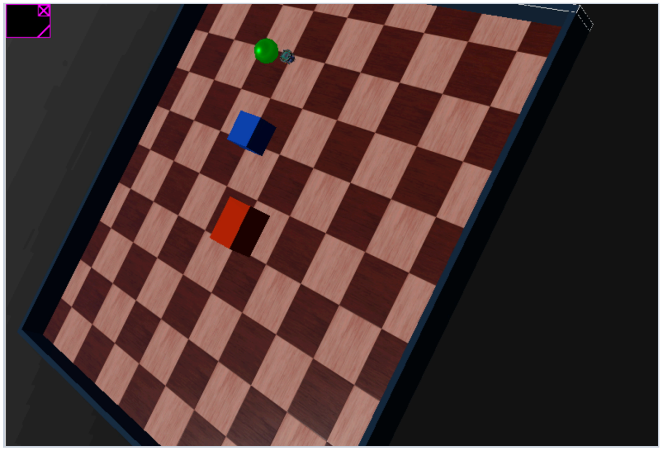


図4　ゴール到達（動画 59 秒、t = 63.2 s）。緑の球の横で停止している。

```
[PAI] t= 54.2s pos=(+0.47,-0.28) goal_distance=0.74m AI=forward IR=(7
[PAI] Waypoint 9/9 reached
[PAI] t= 57.2s pos=(+0.63,-0.16) goal_distance=0.57m AI=forward IR=(6
[PAI] t= 60.2s pos=(+0.72,+0.06) goal_distance=0.34m AI=forward IR=(6
[PAI] t= 63.2s pos=(+0.78,+0.29) goal_distance=0.11m AI=forward IR=(6
[PAI] SUCCESS! Goal reached at t=63.2s
[PAI] Final position (0.775,0.293); distance 0.108m
INFO: 'ai_navigation' controller exited successfully.
```

図5　到達時のコンソールログ（Waypoint 9/9 到達から SUCCESS 行・最終位置まで）。

```
[PAI] AI robot started: supervised k-NN, synthetic labeled examples.
[PAI] Goal: (0.76, 0.40), radius 0.11 m
[PAI] Global waypoints are hand-designed; the local steering policy is learned (k-NN).
[PAI] t=  0.0s pos=(-0.80,-0.34) goal_distance=1.73m AI=forward IR=(70,71)
  ...（3 秒ごとに出力。Waypoint 1/9 ～ 9/9 reached を順に記録）...
[PAI] t= 39.1s pos=(-0.69,-0.41) goal_distance=1.66m AI=turn_right IR=(68,67)
[PAI] Waypoint 7/9 reached
  ...
[PAI] Waypoint 9/9 reached
[PAI] t= 63.2s pos=(+0.78,+0.29) goal_distance=0.11m AI=forward IR=(67,72)
[PAI] SUCCESS! Goal reached at t=63.2s
[PAI] Final position (0.775,0.293); distance 0.108m
```

リスト1　走行ログの抜粋（全文は動画のコンソール画面に収録）。

7. 工夫した点

1. **AI と人の役割分担を明示した設計。**「経路は人が経由地点で設計し、AI は各時刻の行動（前進／左右旋回）だけを判断する」構成にした。どこまでが AI の判断かがログと動画から追えるため、結果の説明可能性と再現性が高い。
2. **往復ルートによる走行距離の拡張。**同じ回廊を 2 回往復してからゴールへ向かうルートにし、AI の判断を約 63 秒・約 5.3 m にわたって検証した。折り返し点では目標方位差がほぼ ±1（真後ろ）になるため、k-NN が旋回を正しく選べるかを確かめる場面になっている。
3. **外部ライブラリ不要の純 Python k-NN。**NumPy や scikit-learn を使わず、Webots 同梱の Python 環境でそのまま動く。環境構築の壁を下げ、コードを読めば推論の仕組みが全て追える。
4. **再現可能な教師データ合成。**教師ルール・サンプル比率・乱数シードを固定したスクリプトで 6,000 例を生成し、評価スクリプトでオフライン精度（94.3%）を確認してから Webots に載せた。
5. **3 次元に圧縮した特徴量設計。**8 個の赤外線センサーを「右前方／左前方の代表値」に集約し、待機時のベースライン（約 75）を補正してから正規化。目標方位差と合わせた 3 次元にしたことで、k-NN の近傍探索が安定した。
6. **AI 判断の周りの安全状態機械。**障害物接近時に旋回を 0.65 秒保持する仕組みを置き、反射的な振動を抑える設計とした（今回は未発動）。
7. **検証可能性を重視したログと動画。**3 秒ごとに位置・ゴール距離・AI の判断・センサー値をログ出力し、動画では 3D 画面とコンソールを同時収録。字幕はすべてログで裏付けられる内容に限定した（倍速なし・66 秒）。

8. 限界と正直に記載しておくこと

- **経路計画は AI ではない。**9 か所の経由地点とゴール座標は人が設定した。AI が自律的に経路を計画したわけではない。
- **自己位置は Webots Supervisor から取得**している。これはシミュレーション専用の特権的な位置情報であり、実機で必要な自己位置推定（オドメトリや SLAM）は実装していない。
- **障害物回避性能は未検証。**設計ルートが障害物の南側を通るため、走行中の赤外線センサー値は常に待機値付近（65–75）で、回避分岐（閾値 100）は一度も発動しなかった。赤・青の障害物は配置したままだが、「AI が障害物を検知して回避した」とは言えない。

- **教師データは合成データ**で、実走行ログや実機データからの学習ではない。オフライン精度 94.3% は教師ルールの再現率であり、走行成功率ではない。
- 検証は 1 つのワールド・1 つのルートで行った。障害物配置や開始位置を変えた場合の頑健性は未確認。

9. 考察と今後の改善

合成データのみで学習した k-NN でも、方位差に基づく「前進か旋回か」の判断は安定しており、折り返し点を含む約 63 秒の走行で一度も迷走せずにゴールへ到達できた。一方で、今回の走行は AI の能力のうち「目標方位への追従」しか使っておらず、特徴量設計の中核である障害物項が評価されていない点が最大の課題である。今後は次の順で発展させたい。

1. **障害物をルート上に置いた検証**：回避分岐と k-NN の障害物項が実際に機能するかを確認し、接触回数・到達時間で評価する。
2. **走行ログからの再学習**：Webots の走行で得たセンサー・行動の組を教師データに加え、合成データとの差を埋める（模倣学習の方向）。
3. **経由地点の自動生成**：占有格子と A* 等で経由地点を計画し、「AI による経路計画＋AI による行動判断」へ拡張する。
4. **Supervisor 依存の解消**：車輪オドメトリや GPS ノイズを含む自己位置推定に置き換え、実機（E-puck 実機や他の小型二輪ロボット）への移行可能性を検討する。
5. **行動の多段化**：速度段階や緩やかな旋回を追加し、走行時間の短縮と滑らかな軌道を目指す。

10. 動画の構成

時間	字幕（2 行）	対応する場面・ログ
0.5–7 s	Physical AI 最終課題／AIによる仮想ロボットの走行制御	走行開始
7.5–17 s	Webots上の小型ロボット E-puck／実機を使わずシミュレーションで検証	シーンツリーの DEF ROBOT E-puck
17.5–27.5 s	教師あり学習のk-NNを活用／センサー情報と目標方向から行動を判断	ログ「supervised k-NN」「IR=…」
28–40 s	設定した経由地点に沿って走行／前進・左右旋回をAIが判定	t = 39.1 s の AI = turn_right
40.5–50.5 s	往復ルートで走行距離を拡張／9か所の経由地点を順番に通過	x 座標が –0.80→+0.57→–0.69→+0.78 と往復
51.5–58 s	全9か所の経由地点を通過／ゴールへ最終接近	「Waypoint 9/9 reached」
58.5–66 s	ゴール到達成功｜シミュレーション時間63.2秒／最終位置とゴールの距離 0.108 m	「SUCCESS! Goal reached at t=63.2s」

編集内容：Webots 画面の収録（61.5 秒）を等速のまま使用し、収録終了操作が映る直前（59.4 秒）でカットして最終フレームを静止させ 66 秒とした。字幕は FFmpeg で焼き込み（SRT も同梱）。ウィンドウのタイトルバーに表示されるファイルパスは個人情報保護のためマスクした。

11. 参考資料

- Webots 公式ドキュメント Tutorial 4: More about Controllers（E-puck の距離センサーとモータ制御）
  `https://github.com/cyberbotics/webots/blob/master/docs/guide/tutorial-4-more-about-controllers.md`
- Webots R2025a E-puck PROTO
  `https://github.com/cyberbotics/webots/blob/R2025a/projects/robots/gctronic/e-puck/protos/E-puck.proto`
- Webots 公式 Python ガイド
  `https://github.com/cyberbotics/webots/blob/master/docs/guide/using-python.md`

本資料・コード・動画には氏名・所属・メールアドレスを含めていない。Webots の基本 API 利用部分を除き、教師データの合成方法、k-NN の実装、経由地点による往復ルート設計、安全状態機械は本課題のために独自に作成したものである。